Programación II

Guía Práctica N.º 8

Librerías aplicadas en Ingeniería

Clase 2: de los datos a las señales

Lenguaje: Python
Librerías: NumPy, Matplotlib y SciPy
Entorno: Visual Studio Code y `.venv`

Edición 2026

Índice

1 Presentación y propósito **1**

2 Indicaciones generales de trabajo **1**

3 Ejercicios obligatorios **2**

1. Tonos de marcación telefónica (DTMF) . . . 2
2. Una señal con la relación señal a ruido deseada . . . 3
3. Decodificar una tecla . . . 3
4. Filtro telefónico . . . 3
5. Aliasing en un conversor . . . 4

4 Ejercicios propuestos **4**

6. Fase y retardo . . . 4
7. Una onda cuadrada hecha de senoidales . . . 5
8. Distinguir frecuencias cercanas . . . 5
9. Modulación de amplitud (AM) . . . 5
10. Buscar portadoras desconocidas . . . 6
11. Ancho de banda ocupado . . . 6
12. Separar canales con filtros pasa-banda . . . 6
13. Eliminar la interferencia de la red eléctrica . . . 6
14. ¿Qué orden de filtro necesito? . . . 7
15. `filtfilt` y `lfilter`: medir el retardo . . . 7
16. El promedio móvil también es un filtro . . . 7
17. ¿Cuánto mejora la SNR un filtro? . . . 8
18. Detectar el tono piloto de FM estéreo . . . 8
19. Decodificador DTMF de un número completo . . . 8
20. Integrador: analizador de espectro con informe . . . 9

5 Lista de comprobación final **9**

1 Presentación y propósito

Esta guía acompaña la segunda clase de la unidad **Librerías aplicadas en Ingeniería**. Se generarán señales, se analizará su espectro y se aplicarán filtros mediante problemas de telecomunicaciones: tonos telefónicos, portadoras, interferencias, canales y modulación.

Es la **Guía Práctica N.º 8** de Programación II y conserva la numeración general de las guías anteriores de C, C++ y Python.

Resultados de aprendizaje

Al finalizar las actividades, se espera que puedas:

- generar señales discretas con NumPy y comprobar condiciones de muestreo;
- agregar ruido controlado y evaluar la relación señal a ruido (SNR);
- analizar espectros para identificar frecuencias, amplitudes y componentes de interés;
- diseñar y aplicar filtros digitales mediante herramientas de SciPy;
- interpretar aliasing, atenuación y resolución en frecuencia;
- presentar resultados en tablas y gráficos, y organizar programas con funciones documentadas y `main()`.

2 Indicaciones generales de trabajo

- **Ejercicios 1 a 5: obligatorios.** Se desarrollarán a partir de lo presentado en la segunda clase de Python.
- **Ejercicios 6 a 20: propuestos.** Son optativos, se recomienda resolverlos; amplían los temas y tienen complejidad creciente.
- Cada tarea se resuelve mediante funciones con su *docstring* (`"""..."""`), y una función `main()` llamada al final del archivo, siguiendo las convenciones de esta unidad.
- Los valores fijos (frecuencias, amplitudes, umbrales y constantes físicas) se definen en mayúsculas al comienzo del programa.
- Los gráficos deben tener título, ejes con unidades, grilla y leyenda cuando se representen varias curvas.
- Los archivos de esta clase pueden guardarse dentro de `GuiaPython/Clase2`; la instalación del entorno se realiza según la **Guía 0 — Ampliación para Python**.

Conceptos en caja negra

Cuando aparezcan conceptos todavía no abordados en otras asignaturas, se proporciona una definición sencilla, la ecuación o función correspondiente y la referencia a la materia donde se estudiarán con mayor profundidad. El objetivo aquí es aplicar correctamente las expresiones, sus unidades y el código.

Función de análisis espectral provista por la cátedra. Utilizá la función `espectro()` **sin modificarla** en todos los ejercicios que requieran analizar el espectro:

```python
import numpy as np
from scipy.fft import fft, fftfreq


def espectro(senal, fs):
    """
    Analizador de espectro por software.

    Recibe: senal (array de muestras) y fs (frecuencia de muestreo [Hz]).
    Devuelve: frecuencias [Hz] y amplitudes (solo frecuencias positivas).
    """
    n = len(senal)
    amplitudes = 2 * np.abs(fft(senal))[: n // 2] / n
    frecuencias = fftfreq(n, 1 / fs)[: n // 2]
    return frecuencias, amplitudes
```

Recordatorio de conceptos de la clase (desarrollados en los capítulos 5 a 7 del documento de estudio):

- **Muestreo:** f_s muestras por segundo; $T_s = 1/f_s$, $N = f_s T$; vector de tiempo `t = np.arange(0, T, 1 / fs)`.
- **Senoidal:** $x(t) = A\sin(2\pi f t)$.
- **Ruido gaussiano:** `rng.normal(0, sigma, len(t))`.
- **Nyquist:** $f_s > 2f_{\max}$.
- **Resolución en frecuencia:** $\Delta f = f_s/N = 1/T$.
- **Filtros:** diseño mediante `signal.butter()` y aplicación con `signal.filtfilt()`.

3 Ejercicios obligatorios

1. Tonos de marcación telefónica (DTMF)

Cuando se presiona una tecla de un teléfono, se transmiten dos tonos simultáneos: uno de *fila* y uno de *columna*. Este sistema se llama DTMF (*Dual-Tone Multi-Frequency*).

	1209 Hz	**1336 Hz**	**1477 Hz**
697 Hz	1	2	3
770 Hz	4	5	6
852 Hz	7	8	9
941 Hz	*	0	#

Por ejemplo, la tecla **5** es la suma de un tono de 770 Hz y uno de 1336 Hz, ambos de amplitud 1.

1. Escribí `tono_dtmf(tecla,t)`, que genere la señal de cualquier tecla. Guardá la tabla en dos diccionarios (`FILA` y `COLUMNA`) que asocien cada tecla con su frecuencia.
2. Generá la tecla 5 con $f_s = 8000\,\text{Hz}$ durante $0.1\,\text{s}$.
3. Verificá con una función que f_s cumple Nyquist para todas las frecuencias de la tabla.
4. Graficá los primeros 20 ms de las teclas 1 y 9 con `plt.subplot(2,1,...)`.

2. Una señal con la relación señal a ruido deseada

CAJA NEGRA — SNR y nivel de ruido

Qué es: la relación señal a ruido (SNR) compara la potencia de la señal con la del ruido. Para simular una SNR dada hay que elegir el desvío del ruido.
Cómo lo usamos:

$$P = \text{np.mean}(x^2), \qquad \text{SNR[dB]} = 10\log_{10}\left(\frac{P_{\text{señal}}}{P_{\text{ruido}}}\right),$$

$$\sigma = \sqrt{\frac{P_{\text{señal}}}{10^{\text{SNR}/10}}}.$$

Lo van a profundizar en: Sistemas de Comunicaciones.

Generá un tono de 1 kHz y amplitud 1, con $f_s = 8000\,\text{Hz}$ durante 1 s.

1. Escribí `sigma_para_snr(senal,snr_db)` que calcule el desvío de ruido necesario.
2. Para SNR $= 20,\ 10,\ 0$ y $-5\,\text{dB}$, agregá el ruido correspondiente (`rng=np.random.default_rng(2)`) y medí la SNR obtenida con `snr_db(senal,ruido)`. ¿Coincide con la deseada?
3. Graficá los primeros 5 ms de las cuatro señales en una figura de 2×2.

3. Decodificar una tecla

Usando la función `tono_dtmf` del ejercicio 1, generá la tecla 8 con $f_s = 8000\,\text{Hz}$ durante 0.1 s y sumale ruido con $\sigma = 0.8$ (`rng=np.random.default_rng(3)`).

Escribí `decodificar_tecla(senal,fs)`, que:

1. calcule el espectro de la señal;
2. encuentre la frecuencia con mayor amplitud entre 650 y 1000 Hz (la fila) y la de mayor amplitud entre 1150 y 1550 Hz (la columna), usando máscaras;
3. elija, para cada una, la frecuencia de la tabla más cercana;
4. devuelva la tecla correspondiente.

Probala con las 12 teclas y verificá que las decodifica todas correctamente. Con 0.1 s de señal, ¿cuál es la resolución en frecuencia? ¿Por qué no hace falta que el pico caiga exactamente en la frecuencia de la tabla?

4. Filtro telefónico

Una línea telefónica transmite voz entre 300 Hz y 3400 Hz. Una señal de prueba contiene tonos de 400 Hz, 1000 Hz y 3000 Hz (amplitud 1 cada uno) y una interferencia de 5500 Hz de amplitud 0.8, muestreados con $f_s = 16000\,\text{Hz}$ durante 0.5 s.

1. Diseñá un pasa-bajos Butterworth de orden 6 con $f_c = 3400\,\text{Hz}$ y aplicalo con `filtfilt`.
2. Informá la amplitud de cada componente antes y después del filtro, y la atenuación en dB de cada una: $20\log_{10}(A_{\text{después}}/A_{\text{antes}})$.
3. Construí la figura de 2×2 (tiempo y frecuencia, antes y después). En los gráficos temporales mostrá solo 10 ms.

¿Se atenúa algo la componente de 3000 Hz? ¿Por qué?

5. Aliasing en un conversor

Un conversor analógico-digital muestrea a $f_s = 10\,\text{kHz}$. Llega un tono de 7 kHz.

CAJA NEGRA — Frecuencia aparente por aliasing

$$f_{\text{aparente}} = \left| f - f_s \cdot \text{redondeo}\left(\frac{f}{f_s}\right) \right|.$$

En Python: `abs(f-fs*round(f/fs))`.

1. Calculá con la fórmula dónde va a aparecer el tono.
2. Generá el tono muestreado a 10 kHz durante 0.1 s y verificá con `espectro()` y `np.argmax` la frecuencia aparente.
3. Repetí para tonos de 1, 3, 6, 9, 11 y 14 kHz: armá una tabla con la frecuencia real, la calculada con la fórmula y la medida en el espectro.
4. ¿Cuál es la menor f_s que permite representar correctamente todos esos tonos?

4 Ejercicios propuestos

6. Fase y retardo

CAJA NEGRA — Fase

Qué es: la fase indica en qué punto de su ciclo arranca una senoidal. Una diferencia de fase entre dos señales de la misma frecuencia equivale a un retardo en el tiempo.
Cómo lo usamos (φ en radianes):

$$x(t) = A\sin(2\pi f t + \varphi), \qquad \text{retardo} = \frac{\varphi}{2\pi f}.$$

Lo van a profundizar en: Señales y Sistemas.

Una antena recibe la misma señal de 1 kHz por dos caminos. Generá las dos señales con $f_s = 100\,\text{kHz}$: la directa con $\varphi = 0$ y la reflejada con $\varphi = \pi/2,\ \pi$ y $3\pi/2$ (tres casos). Para cada caso:

1. calculá el retardo en microsegundos;
2. graficá las dos señales superpuestas (2 ms);
3. calculá la amplitud de la suma de las dos (el máximo de `np.abs`). ¿Qué pasa con $\varphi = \pi$? Este fenómeno, llamado *interferencia destructiva*, es una de las causas de los desvanecimientos de señal.

7. Una onda cuadrada hecha de senoidales

CAJA NEGRA — Serie de Fourier de una onda cuadrada

Qué es: una onda cuadrada se puede construir sumando senoidales de frecuencias impares (sus armónicos).
Cómo lo usamos (f_0: frecuencia fundamental; $k = 1, 3, 5, 7, \ldots$):

$$x(t) = \frac{4}{\pi} \sum_{k \text{ impar}} \frac{\sin(2\pi k f_0 t)}{k}.$$

Lo van a profundizar en: Señales y Sistemas.

Con $f_0 = 100\,\text{Hz}$ y $f_s = 20\,\text{kHz}$, construí la onda sumando 1, 3, 10 y 50 armónicos impares (función `onda_cuadrada(t,f0,cantidad_armonicos)`). Graficá las cuatro aproximaciones en 20 ms. Luego, calculá el espectro de la de 10 armónicos y verificá con una máscara en qué frecuencias aparecen picos y que su amplitud es $4/(\pi k)$.

8. Distinguir frecuencias cercanas

Dos emisoras transmiten tonos de 100 Hz y 102 Hz, de amplitud 1, muestreados con $f_s = 1000\,\text{Hz}$.

1. Calculá la resolución en frecuencia para registros de 0.25 s, 0.5 s, 1 s y 2 s.
2. Para cada duración, calculá el espectro e informá en qué frecuencias, entre 95 y 107 Hz, la amplitud supera 0.5.
3. Graficá los cuatro espectros (solo entre 90 y 112 Hz) en una figura de 2×2.

¿A partir de qué duración aparecen los dos tonos en sus frecuencias correctas? ¿Qué pasa con 0.25 s? ¿Coincide con lo que predice la resolución?

9. Modulación de amplitud (AM)

CAJA NEGRA — Modulación AM

Qué es: la forma más antigua de transmitir información por radio: la amplitud de una portadora de alta frecuencia varía siguiendo el mensaje.
Cómo lo usamos (f_c: portadora; f_m: frecuencia del mensaje; m: índice de modulación, entre 0 y 1):

$$x(t) = (1 + m\cos(2\pi f_m t))\cos(2\pi f_c t).$$

Su espectro tiene tres picos: la portadora en f_c (amplitud 1) y dos bandas laterales en $f_c - f_m$ y $f_c + f_m$ (amplitud $m/2$ cada una).
Lo van a profundizar en: Sistemas de Comunicaciones.

Con $f_c = 1000\,\text{Hz}$, $f_m = 50\,\text{Hz}$, $m = 0.5$ y $f_s = 10\,\text{kHz}$ durante 1 s:

1. generá la señal AM y graficá 60 ms, junto con la envolvente $1 + m\cos(2\pi f_m t)$ y su opuesta;
2. calculá el espectro y verificá con una máscara las frecuencias y amplitudes de los tres picos;
3. repetí con $m = 1$ y $m = 0.2$. ¿Qué cambia en el espectro?

10. Buscar portadoras desconocidas

La cátedra entrega esta función, que simula lo que recibe una antena:

```
def senal_misteriosa(t, semilla):
    """Tres portadoras de frecuencia y amplitud desconocidas, más ruido."""
    rng = np.random.default_rng(semilla)
    frecuencias = rng.choice(np.arange(500, 4500, 10), 3, replace=False)
    amplitudes = rng.uniform(0.3, 1.0, 3)
    senal = np.zeros(len(t))
    for f, a in zip(frecuencias, amplitudes):
        senal = senal + a * np.sin(2 * np.pi * f * t)
    return senal + rng.normal(0, 0.8, len(t))
```

Con $f_s = 10\,\text{kHz}$ y 1 s de señal, y sin mirar las variables internas de la función, escribí un programa que encuentre las tres portadoras y sus amplitudes a partir del espectro. Usá un umbral que dependa del piso de ruido: por ejemplo, 10 veces la mediana de las amplitudes. Probalo con las semillas 1, 2 y 3. Verificá tus resultados solo al final, imprimiendo las frecuencias verdaderas dentro de la función.

11. Ancho de banda ocupado

El ancho de banda ocupado de una señal es el rango de frecuencias donde su espectro tiene energía significativa. Una forma simple de estimarlo: el rango entre la menor y la mayor frecuencia cuya amplitud supera el 1 % del máximo.

1. Escribí `ancho_banda_ocupado(frecuencias,amplitudes,fraccion)`.
2. Calculalo para la señal AM del ejercicio 9 con $f_m = 50,\ 100$ y 200 Hz (sin ruido). ¿Qué relación hay entre el ancho de banda y f_m?
3. Calculalo para la onda cuadrada del ejercicio 7 con 1, 10 y 50 armónicos. ¿Por qué una onda cuadrada "perfecta" necesitaría un ancho de banda infinito?

12. Separar canales con filtros pasa-banda

En una transmisión con multiplexación por división de frecuencia (FDM), varios canales comparten el medio, cada uno en su propia banda. Una señal contiene cuatro canales, representados por tonos de 1000, 2000, 3000 y 4000 Hz con amplitudes 1.0, 0.7, 0.5 y 0.3, más ruido con $\sigma = 0.2$ (`rng=np.random.default_rng(12)`), con $f_s = 16000\,\text{Hz}$ durante 0.5 s.

1. Escribí `extraer_canal(senal,f_central,ancho,fs)`, que aplique un pasa-banda de orden 4 entre $f_{\text{central}} - \text{ancho}/2$ y $f_{\text{central}} + \text{ancho}/2$.
2. Extraé cada canal con un ancho de 400 Hz e informá su amplitud recuperada (desde el espectro).
3. Para el canal de 3000 Hz, medí cuánto quedó de los otros tres canales, en dB respecto del canal extraído.
4. Graficá el espectro original y el de los cuatro canales extraídos (figura de 5 gráficos, uno debajo del otro).

13. Eliminar la interferencia de la red eléctrica

Un sensor transmite una señal útil de 17 Hz (amplitud 1), pero el cableado le induce la interferencia de la red eléctrica: 50 Hz (amplitud 0.6) y su tercer armónico, 150 Hz (amplitud 0.3). La señal se muestrea con $f_s = 1000\,\text{Hz}$ durante 2 s.

1. Diseñá dos filtros rechaza-banda de orden 4: uno entre 45 y 55 Hz y otro entre 145 y 155 Hz. Aplicalos en cascada (la salida de uno es la entrada del otro).
2. Informá la amplitud de las tres componentes antes y después.
3. Calculá el error máximo entre la señal limpia y la señal útil ideal, descartando las primeras y últimas 100 muestras.
4. ¿Por qué no alcanzaría un pasa-bajos con $f_c = 30\,\text{Hz}$ si la señal útil tuviera componentes de hasta 100 Hz?

14. ¿Qué orden de filtro necesito?

CAJA NEGRA — Respuesta en frecuencia

Cómo lo usamos: `frecuencias,h=signal.freqz(b,a,worN=4000,fs=fs)` y ganancia en dB: `20*np.log10(np.abs(h))`.

Para un pasa-bajos Butterworth con $f_c = 1000\,\text{Hz}$ y $f_s = 16000\,\text{Hz}$:

1. calculá, para órdenes de 1 a 10, la atenuación en 2000 Hz (el doble de f_c);
2. mostrá los resultados en una tabla y encontrá el menor orden que atenúa al menos 40 dB en 2000 Hz;
3. graficá la respuesta en dB de los órdenes 2, 4 y 8, con una línea vertical en f_c y otra en $2f_c$.

Verificá que todos los órdenes atenúan 3 dB en la frecuencia de corte. (Recordá que `filtfilt` aplica el filtro dos veces y duplica la atenuación en dB; aquí se pide la de una sola pasada.)

15. `filtfilt` y `lfilter`: medir el retardo

Un pulso llega al receptor. Generá con $f_s = 1000\,\text{Hz}$, durante 1 s, un pulso gaussiano centrado en $t = 0.5\,\text{s}$:

$$x(t) = \exp\left(-\left(\frac{t - 0.5}{0.01}\right)^2\right).$$

1. Filtralo con un pasa-bajos de orden 4 y $f_c = 30\,\text{Hz}$ usando `filtfilt` y usando `lfilter`.
2. Con `np.argmax`, encontrá el instante del máximo de cada salida y calculá el retardo respecto del pulso original, en milisegundos.
3. Graficá el pulso original y las dos salidas superpuestas, entre 0.4 s y 0.6 s.
4. Repetí con $f_c = 15\,\text{Hz}$ y $f_c = 60\,\text{Hz}$. ¿Cómo cambia el retardo de `lfilter`?

¿Por qué un receptor en tiempo real no puede usar `filtfilt`?

16. El promedio móvil también es un filtro

CAJA NEGRA — Promedio móvil como filtro

Cómo lo usamos: el promedio de N muestras se aplica con `np.convolve(x,np.ones(N)/N, mode="same")`. Es un filtro con coeficientes `b=np.ones(N)/N` y `a=[1]`; su respuesta se puede ver con `signal.freqz(b,[1],worN=4000,fs=fs)`.

Con $f_s = 1000\,\text{Hz}$:

1. aplicá un promedio móvil de $N = 10$ muestras a la señal de 100 Hz + 250 Hz + ruido de la clase ($\sigma = 0.3$, semilla 42) y compará el espectro resultante con el del pasa-bajos Butterworth de orden 4 y $f_c = 150\,\text{Hz}$;
2. graficá la respuesta en frecuencia del promedio móvil para $N = 5,\ 10$ y 20;
3. verificá que la respuesta del promedio móvil tiene ceros (atenuación muy grande) en las frecuencias múltiplos de f_s/N.

¿Qué ventajas y desventajas tiene el promedio móvil frente al Butterworth?

17. ¿Cuánto mejora la SNR un filtro?

Un tono de 100 Hz (amplitud 1) llega con ruido de $\sigma = 1.5$, muestreado con $f_s = 1000\,\text{Hz}$ durante 2 s (`rng=np.random.default_rng(17)`).

1. Medí la SNR de la señal recibida (usando la caja negra del ejercicio 2).
2. Aplicá pasa-bandas de orden 4 centrados en 100 Hz con anchos de 100, 40, 20 y 10 Hz.
3. Para cada uno, medí la SNR después de filtrar. Para separar señal de ruido, filtrá por separado el tono y el ruido (el filtro es lineal: filtrar la suma es sumar las partes filtradas).
4. Graficá la mejora de SNR en función del ancho de banda del filtro.

¿Qué relación aproximada encontrás entre la reducción del ancho de banda y la mejora de SNR? Relacionalo con el piso de ruido térmico (kTB).

18. Detectar el tono piloto de FM estéreo

Las emisoras de FM estéreo transmiten un tono piloto de 19 kHz de baja amplitud, que el receptor usa para saber que la transmisión es estéreo. Con $f_s = 48\,\text{kHz}$ y 0.05 s de señal:

1. escribí `detecta_piloto(senal,fs,umbral)`, que busque el máximo del espectro entre 18.9 y 19.1 kHz y devuelva `True` si supera el umbral;
2. simulá 50 recepciones (semillas 0 a 49) de un piloto de amplitud A más ruido de $\sigma = 1$, y medí el porcentaje de detecciones correctas para $A = 0.05,\ 0.1,\ 0.2$ y 0.4, con umbral 0.1;
3. simulá también 50 recepciones sin piloto (solo ruido) y medí el porcentaje de falsas alarmas;
4. graficá el porcentaje de detección en función de A.

¿Qué pasa con las falsas alarmas si bajás el umbral a 0.05? Este compromiso entre detectar y no dar falsas alarmas es central en todo sistema de recepción.

19. Decodificador DTMF de un número completo

Un usuario marca un número de 5 dígitos. Cada tecla dura 0.1 s y entre teclas hay 0.05 s de silencio. La señal se muestrea a 8000 Hz y llega con ruido de $\sigma = 0.5$.

1. Escribí `generar_marcacion(numero,fs,rng)`, que construya la señal completa del número (usá `np.concatenate`, buscando cómo funciona en la documentación).

2. Escribí `segmentar(senal,fs)`, que separe los tramos de cada tecla sabiendo la duración de teclas y silencios.
3. Usando `decodificar_tecla` del ejercicio 3, decodificá el número completo.
4. Probalo con los números `"35812"` y `"90467"`, con `rng=np.random.default_rng(19)`.
5. **Desafío:** hacé que `segmentar` detecte los tramos sin conocer los tiempos, calculando la energía de la señal en ventanas de 10 ms y buscando dónde supera un umbral.

20. Integrador: analizador de espectro con informe

Escribí un analizador de espectro que genere un informe completo de cualquier señal. El programa debe tener, al menos, estas funciones:

```
def espectro(senal, fs): ...
    # Provista por la catedra; utilizar la implementacion anterior.
def espectro_db(amplitudes): ...
    # 20 * log10(amplitud / maximo)
def piso_ruido(amplitudes): ...
    # Mediana de las amplitudes
def picos(frecuencias, amplitudes, margen_db): ...
    # Picos a mas de margen_db sobre el piso
def ancho_banda_ocupado(frecuencias, amplitudes, fraccion): ...
def informe(senal, fs, nombre): ...
    # Imprime el informe y grafica
def main(): ...
```

El informe de cada señal debe indicar: frecuencia dominante, lista de picos detectados (frecuencia y amplitud), piso de ruido, SNR estimada del pico dominante respecto del piso (en dB) y ancho de banda ocupado. El gráfico debe mostrar la señal en el tiempo y el espectro en dB (normalizado al máximo), con una línea horizontal en el nivel de detección.

Probalo con tres señales, todas con $f_s = 10\,\text{kHz}$ y 1 s de duración:

1. la tecla 7 de DTMF con $\sigma = 0.3$;
2. la señal AM del ejercicio 9 ($m = 0.5$) con $\sigma = 0.1$;
3. la señal misteriosa del ejercicio 10 con semilla 4.

5 Lista de comprobación final

Antes de dar por terminada la guía

- ☐ Se utiliza el entorno Python de la unidad y están disponibles NumPy, Matplotlib y SciPy.
- ☐ Los ejercicios se organizan en archivos `.py` identificables dentro de `GuiaPython/Clase2`.
- ☐ Cada programa utiliza funciones documentadas y una llamada final a `main()`.
- ☐ Se utiliza `espectro()` provista por la cátedra, sin modificarla, donde corresponde.
- ☐ Las frecuencias, amplitudes, tiempos y demás magnitudes emplean las unidades correctas.
- ☐ Se respetan las frecuencias de muestreo y las condiciones de Nyquist solicitadas.
- ☐ Los resultados espectrales, la SNR y los filtros se verificaron según cada consigna.

- ☐ Los gráficos muestran título, ejes con unidades, grilla y leyenda cuando corresponde.
- ☐ Se respondieron las preguntas de análisis e interpretación de los ejercicios realizados.